

Mouseless

Navigation souris entièrement au clavier sous Linux

Projet personnel — EPITA GISTRE

Stack : Rust / X11 / AT-SPI / NixOS / i3wm

Résumé du projet

Mouseless est un outil système bas niveau écrit en Rust, conçu pour permettre une navigation souris entièrement au clavier sous Linux (NixOS + i3wm). Inspiré de l'extension Vimium pour Firefox et du mode hint de Helix, il étend ce paradigme à l'ensemble du bureau.

Lorsqu'un raccourci clavier est déclenché, l'outil capture le contenu de l'écran, détecte tous les éléments textuels via l'API d'accessibilité AT-SPI (avec fallback OCR pour les apps non-compatibles), puis affiche une overlay transparente sur laquelle chaque élément est labelisé par une combinaison de 2 touches. L'utilisateur tape ces touches et le curseur se téléporte instantanément à la cible.

Contexte et motivation

Sur un environnement tiling window manager comme i3, l'objectif est de minimiser au maximum les interactions avec la souris. Des outils comme warpd ou chelleport existent mais présentent des limitations :

- **warpd** : mode grille uniquement, pas de détection de texte intelligente
- **chelleport** : écrit en Haskell avec FFI C++, X11 uniquement, latence OCR ~1s, pas de Wayland

Mouseless vise à corriger ces points : Rust natif, AT-SPI en priorité (quasi-instantané), OCR en fallback optimisé, et une architecture pensée pour évoluer vers Wayland.

Plan de développement

Phase 1 — Fondations

1.1 — Capture d'écran

- **Crate** : x11rb pour capturer le contenu de l'écran via X11
- Retourner une image brute (buffer pixels) prête à être analysée
- Optimisation : ne capturer que les zones visibles nécessaires

1.2 — Raccourci global

- **Mécanisme** : XGrabKey via xcb pour intercepter une touche indépendamment du focus
- Définir le raccourci d'activation (ex : Super+Space)
- Concevoir une event-loop propre dès le départ pour faciliter les extensions futures

1.3 — Déplacement souris hjkl

- **API** : XTestFakeMotionEvent pour déplacer la souris par événements synthétiques
- Mapper les touches configurables (hjkl ou I;J) en mouvements relatifs
- Step normal + step large avec Shift

Phase 2 — Overlay transparente

2.1 — Fenêtre overlay

- **Type** : Fenêtre X11 fullscreen, sans decorations, always-on-top
- Transparence via ARGB visual + extension Composite
- Elle capture tous les inputs clavier le temps de la sélection, puis disparaît

2.2 — Rendu des hints

- **Crate** : cairo-rs pour le rendu vectoriel sur l'overlay
- Chaque hint = rectangle semi-transparent coloré + 2 lettres
- Algorithme anti-overlap : éviter que deux hints se chevauchent visuellement
- Contraste garanti : fond adaptatif selon la luminosité du fond détecté

Phase 3 — Détection de texte

3.1 — AT-SPI via D-Bus (méthode principale)

- **Principe** : Chaque app GTK/Qt/Firefox expose un arbre d'accessibilité sur D-Bus
- Utiliser zbus (crate Rust pure D-Bus) pour parcourir cet arbre
- Extraire : texte, coordonnées écran, rôle (lien, bouton, label, champ)
- **Latence** : quelques millisecondes, aucune analyse image nécessaire

3.2 — Génération des hints

- Assigner une combinaison de 2 lettres unique à chaque élément détecté
- Tri par position haut-gauche → bas-droite pour des combos prévisibles
- Matching progressif : filtrer en temps réel à chaque touche tapée

3.3 — Fallback OCR

- **Cibles** : apps Electron, terminaux, jeux — tout ce qui n'expose rien via AT-SPI
- **Crate** : tesseract-rs ou appel subprocess Tesseract

- Pipeline : screenshot → crop zones pertinentes → OCR → coordonnées → hints
- Optimisation : PaddleOCR envisagé si la latence Tesseract reste trop élevée

Phase 4 — Interactions avancées

4.1 — Clic et actions souris

- **API** : XTestFakeButtonEvent pour simuler clic gauche, droit, double-clic
- Raccourci dédié pour cliquer sur place (sans ouvrir l'overlay)
- Digit + espace pour répéter le clic N fois (ex : 2 + space = double-clic)

4.2 — Sélection de zone / drag

- Mode sélection : activation par raccourci dédié
- Premier hint = point de départ du drag
- Deuxième hint = point d'arrivée
- **Simulation** : MouseDown → déplacement → MouseUp pour sélectionner une zone de texte
- Copie automatique dans le clipboard via arboard

Phase 5 — Configuration & polish

5.1 — Fichier de configuration

- **Format** : TOML + serde pour la désérialisation
- Paramètres : raccourcis, couleurs hints, step size hjkl, touches utilisées
- Rechargement à chaud sans redémarrer le daemon

5.2 — Architecture daemon

- Process qui tourne en background, activé par raccourci depuis i3
- **Avantage** : latence minimale à l'activation vs lancement oneshot à chaque fois
- Communication par socket Unix ou signal pour déclencher les modes

Stack technique

Besoin	Crate Rust	Rôle
X11 (fenêtre, events)	x11rb	Capture écran, overlay, input
D-Bus / AT-SPI	zbus	Requêter l'arbre d'accessibilité
Rendu overlay	cairo-rs	Dessiner les hints à l'écran
OCR fallback	tesseract-rs	Détecter texte dans apps non-AT-SPI
Config fichier	toml + serde	Raccourcis, couleurs, pas hjkl

Clipboard	arboard	Copier la sélection automatiquement
-----------	---------	-------------------------------------

Limitations connues et pistes futures

- **X11 uniquement (Phase 1-4)** : Wayland nécessite wlr-layer-shell + wlr-virtual-pointer, prévu en Phase 6
- **Apps Electron** : AT-SPI souvent mal exposé, OCR fallback obligatoire
- **Jeux / OpenGL** : aucune accessibilité disponible, OCR ou grille fixe uniquement
- **Latence OCR** : Tesseract ~500ms-1s sur écran dense, optimisation PaddleOCR envisagée

Pertinence pour le blog GISTRE / PBN

Ce projet couvre un spectre technique directement dans le domaine GISTRE :

- Interaction bas niveau avec le serveur X11 (XTest, XGrabKey, Composite)
- Communication inter-processus via D-Bus (protocole système Linux)
- Rendu graphique d'une fenêtre overlay sans toolkit haut niveau
- Simulation d'événements matériels (souris, clavier) au niveau kernel/X
- Analyse et parcours d'arbre d'accessibilité AT-SPI

L'article de blog documentera les choix d'architecture, les difficultés rencontrées (notamment la gestion de l'overlay transparente et l'intégration AT-SPI en Rust), et les benchmarks de latence entre les approches AT-SPI et OCR.